

TP02 – Compteurs

Questions 1. L'horloge du système est fixée à 100MHz. Combien de période faut-il compter pour attendre 2 secondes ? Combien de bits faut-il au minimum pour représenter cette valeur ?

$$f=100 \text{ Mhz}=100\,000\,000 \text{ Hz}$$

Cela veut donc dire qu'en une seconde nous avons 100 000 000 périodes d'horloge

Pour attendre 2 secondes il faut donc $100\,000\,000 \times 2 = 200\,000\,000$ périodes d'horloge.

On cherche le plus petit entier n tel que :

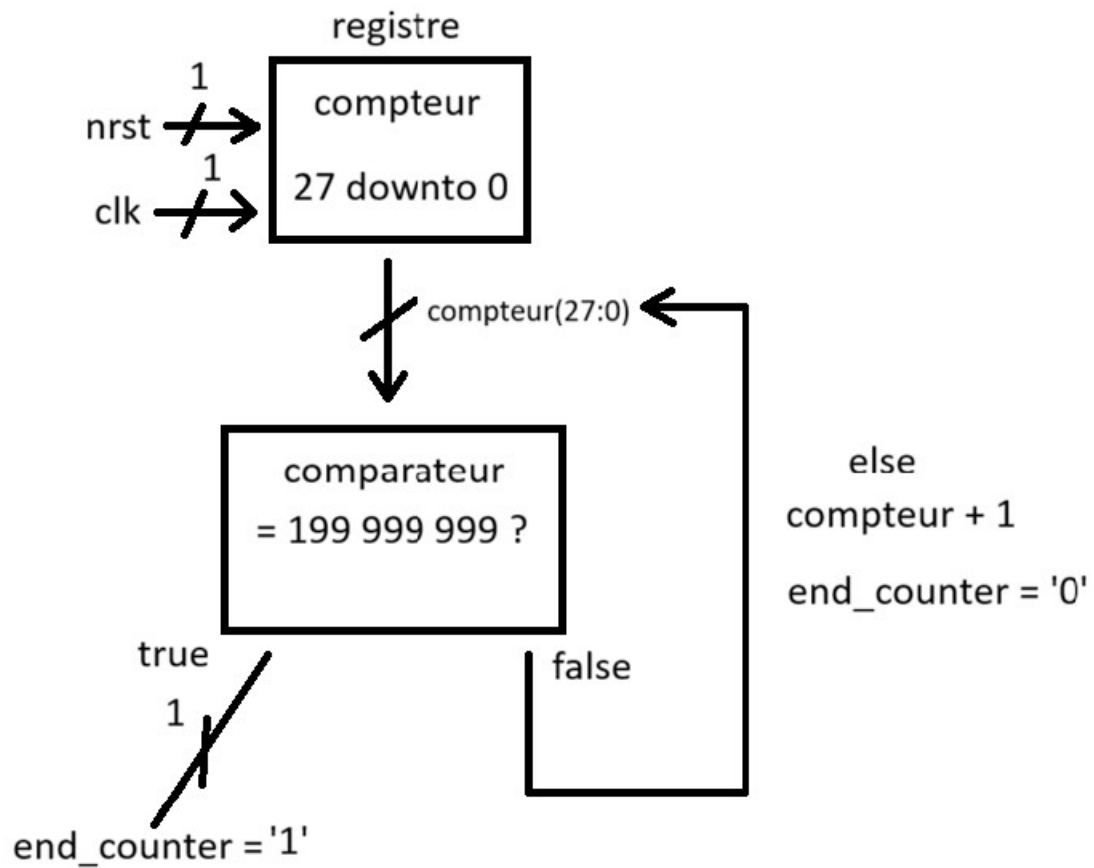
$$2^n \geq 200\,000\,000$$

$$2^{27}=134\,217\,728=134217728 \Rightarrow \text{pas assez, on doit aller jusqu'à } 200\,000\,000$$

$$2^{28}=268\,435\,456=268435456 \Rightarrow \text{suffisant}$$

Il faut donc **28 bits**.

2. Dessinez le schéma RTL de ce compteur. Si le compteur atteint la valeur calculée précédemment, un signal end_counter passe à 1, sinon end_counter vaut 0. N'oubliez pas de mettre sur chaque signal son nombre de bits. Commencez par réaliser une boucle d'incrémentation : +1 à chaque coup d'horloge.



3. Ajoutez une condition pour que le compteur soit remis à 0 lorsqu'il a atteint la valeur souhaitée.

Lorsque compteur = 199 999 999, on affecte end_counter = '1' il faut aussi penser à faire : counter <= (others => '0');

Ainsi le compteur recommence automatiquement après 2 secondes.

4. Listez les signaux d'entrée, de sortie et les signaux internes de votre architecture.

Input :

-clk : std_logic

-nrst : std_logic

Output :

end_counter: std_logic

Signaux interne :

-compteur : unsigned(27 downto 0)

5. Ecrivez à présent le compteur en VHDL en suivant le schéma RTL, faites attention de bien faire correspondre les noms des signaux de votre code VHDL avec ceux de votre schéma RTL.

Fait

6. Ecrivez un fichier de testbench pour tester votre design.

Fait

7. Lancez une simulation. Que devez-vous observer sur votre chronogramme pour vérifier que votre design est valide ?

Dans un premier temps, pour les besoins de la simulation nous avons besoin de l'accélérer, en effet il sera très long de générer 200 000 000 coups d'horloge pour voir ce qu'il s'y passe, c'est pourquoi pour cette étape d'analyse, nous allons définir que notre compteur devra s'incrémenter de +1 jusqu'à 7 et non 200 000 000, nous fixons donc la valeur de notre generic MAX_COUNT = « 111 » = 7bin.

Je dois observer plusieurs choses pour affirmer que le design est valide :

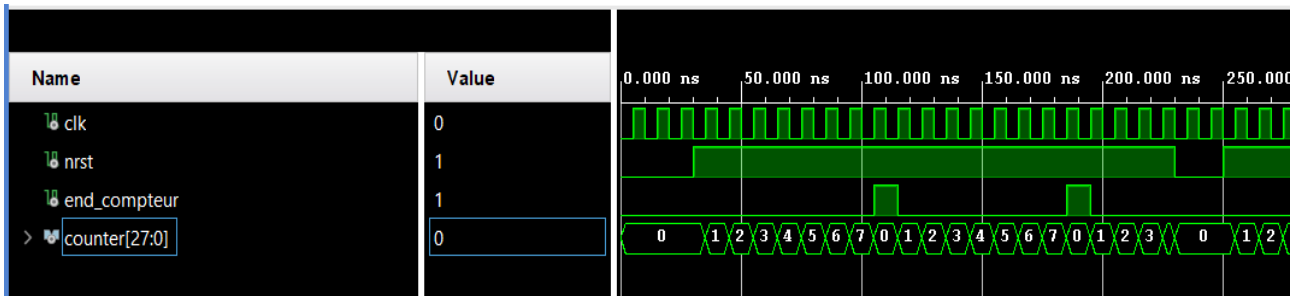
-compteur s'incrémente de +1 à chaque front montant

-Le signal end_counter passe à '1' lorsque notre compteur a atteint la valeur souhaitée (7decimal = « 111 »binaire) puis le compteur recommence.

-Lorsque notre reset négatif = '0', le compteur et la sortie sont à 0.

On remarque bien ici que la sortie end_counter revient à 0 lorsque notre compteur a atteint la valeur 7(dec) soit (« 111 » binaire), nous l'avons initialisé dans notre description matérielle.

```
elseif rising_edge(clk) then
  if counter = "111" then
    end_compteur <= '1';
    counter <= (others => '0');
  else
```

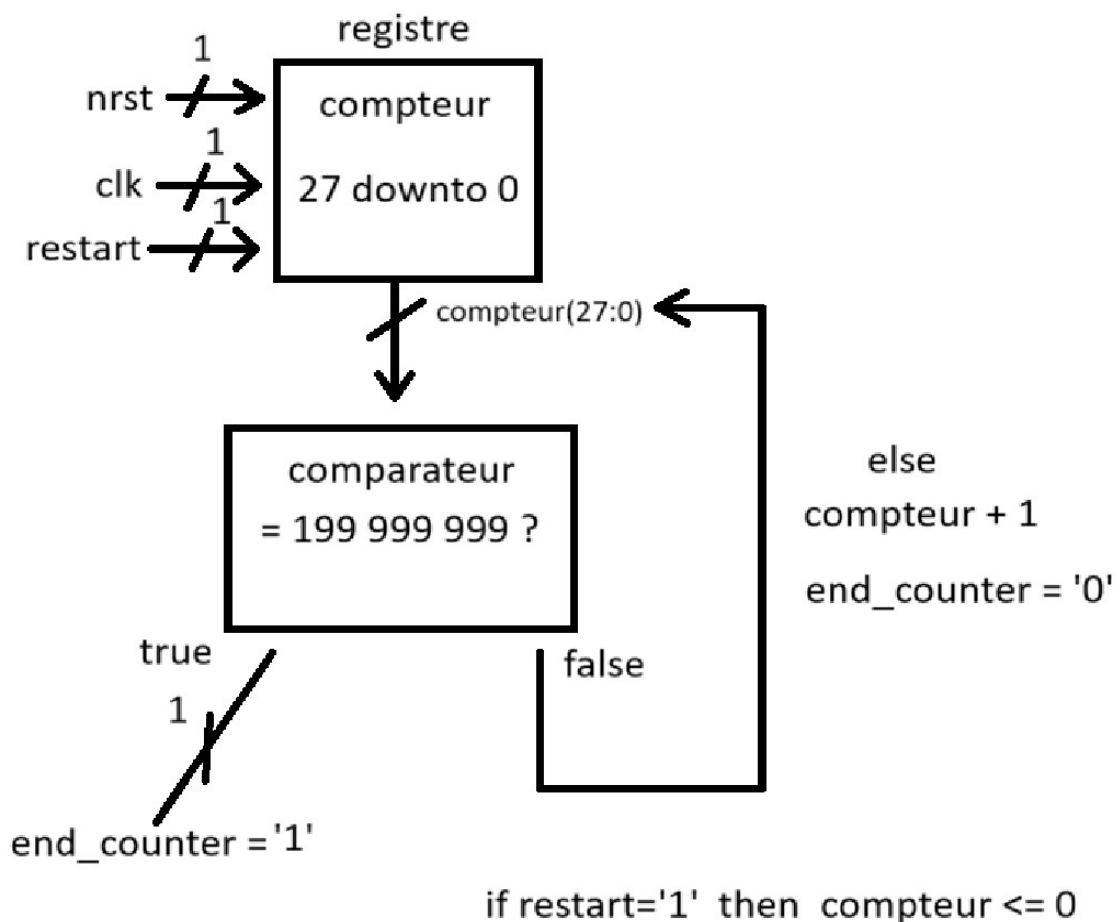


Le signal d'horloge est correcte, le reset negatif à l'état bas maintient le counter à 0. Lorsque ce reset négatif passe à l'état haut le counter commence à compter et le signal end_compteur passe à '1' lorsque le counter atteint sa valeur max.

- Associez une LED avec le signal de teste d'arrêt du compteur. Pour cela, il faudra ajouter une sortie et la relier à une broche d'une LED dans le fichier de contrainte (.xdc). La LED sera alors allumée pendant seulement un coup d'horloge :

```
#set_property -dict { PACKAGE_PIN N15 IOSTANDARD LVCMOS33 } [get_ports { led0_r }]; #IO L21P_T3_DQS_AD14P_35 S
set_property -dict { PACKAGE_PIN G14 IOSTANDARD LVCMOS33 } [get_ports { end_compteur }]; #IO 0_35 Sch=led1 b
```

- Modifiez le schéma RTL du compteur pour ajouter une remise à 0 lorsqu'un signal restart est à 1. Ajoutez la logique nécessaire pour que la LED clignote telle que : allumée 2s, éteinte 2s.



10. Faites les mises à jour nécessaires sur le code VHDL pour correspondre au nouveau schéma. Le signal restart sera une entrée du design.

Voici l'entrée « restart » dans notre design :

```
Port ( clk : in STD_LOGIC;  
      nrst : in STD_LOGIC;  
      restart: in STD_LOGIC;  
      end_counter : out STD_LOGIC;  
      led_o: out STD_LOGIC);
```

Pour faire en sorte que notre LED clignote telle que 2s allumée / 2s éteinte j'ai initialisé un signal « led » qui prend « not led » une fois le compteur max atteint (soit 2 secondes) :

```
elsif counter = MAX_COUNT then --  
    end_counter <= '1';  
    led <= not led;  
    counter <= (others => '0');
```

La led passera donc par exemple de 0 → 1. Ensuite le compteur est réinitialisé et re-comptera 2s pour ensuite rechanger la valeur de led qui passera cette fois-ci à 1 → 0 et ainsi de suite .

Ainsi, la led alternera d'état à chaque fois que le counter atteindra sa valeur max, soit toutes les 2 secondes.

J'ai ensuite affecté ce signal led à un port de sortie led_o dans mon entity :

```
) entity counter is  
    generic (  
        MAX_COUNT : unsigned(27 downto 0)  
    );  
  
    Port ( clk : in STD_LOGIC;  
          nrst : in STD_LOGIC;  
          restart: in STD_LOGIC;  
          end_counter : out STD_LOGIC;  
          led_o: out STD_LOGIC);  
end counter;  
  
) architecture rtl of counter is  
  
    signal counter: unsigned(27 downto 0);  
    signal led: std_logic;  
  
    begin  
  
        led_o <= led;  
  
    process(clk, nrst)  
  
        begin  
  
            -- ...  

```

11. Associez la nouvelle entrée restart à un bouton :

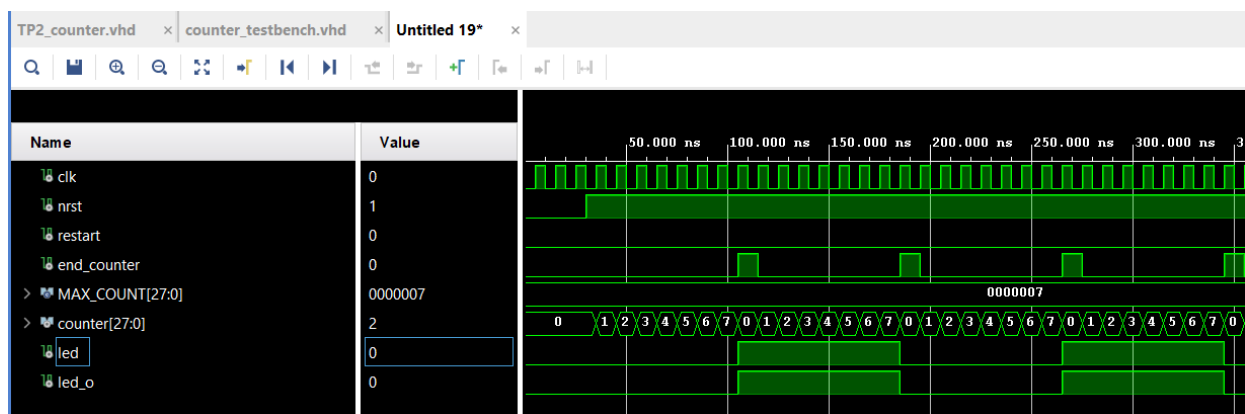
```
# Buttons
set_property -dict { PACKAGE_PIN D20   IOSTANDARD LVCMOS33 } [get_ports { restart }]; #IO_L4N_T0_35 Sch=btn[0]
```

12. Mettez à jour votre testbench puis vérifiez votre design avec une simulation. Quels sont les signaux que vous devez observer ?

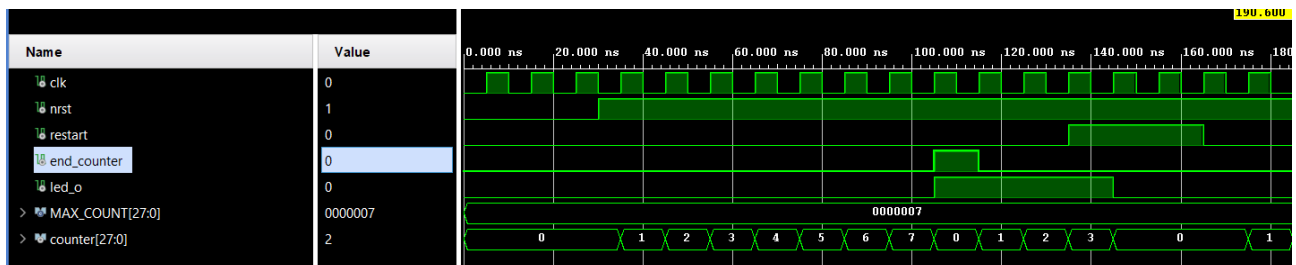
On doit observer les signaux de clk, nrst, restart, counter, end_counter, et led/led_o.

Ici on observe bien que la sortie led alterne entre l'état haut et bas à raison de 7 coups d'horloge (notre max counter). On testera ensuite en condition réelle sur l'ILA et la carte pendant 200 000 000 coups d'horloge.

De plus, on remarque que le signal de test d'arrêt du compteur passe à 1 lorsque le compteur a atteint sa valeur max indiquant bien l'arrêt du compteur comme prévu :



Cette fois-ci, lorsque je mets restart à '1' on remarque bien que le compteur se réinitialise au coup d'horloge suivant (en effet, ce n'est pas instantané car le restart est synchrone contrairement au reset qui lui est asynchrone et donc réinitialise le système instantanément :

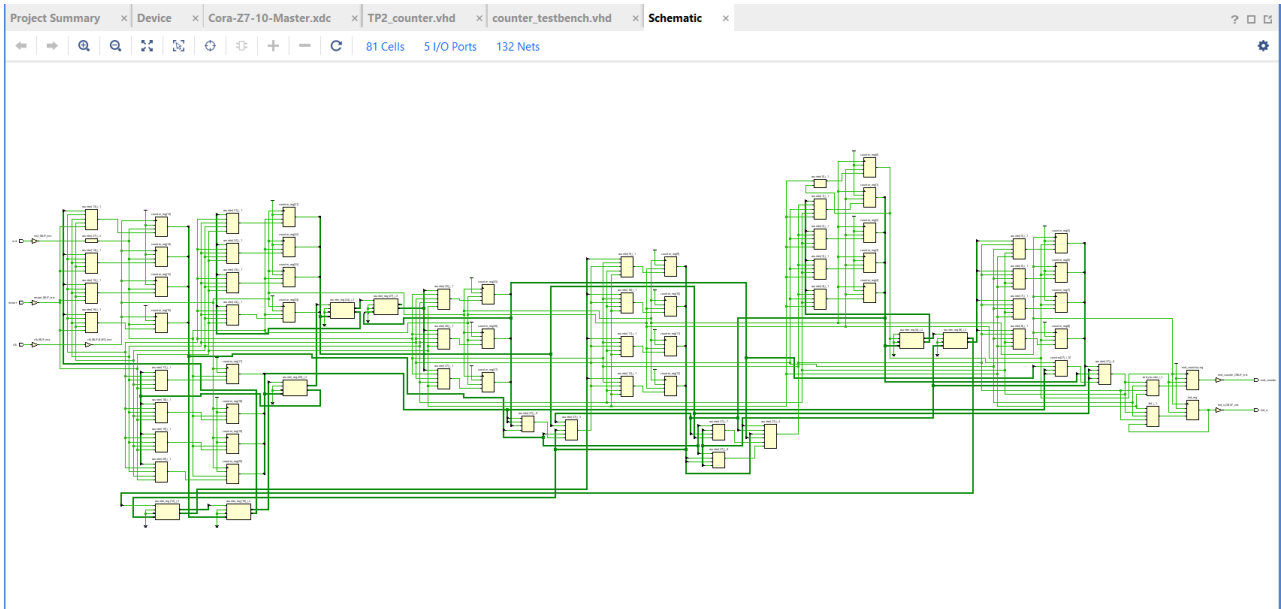


Le compteur passe à 3 puis 0 interrompant la led dans son état haut, la forçant à l'état bas.

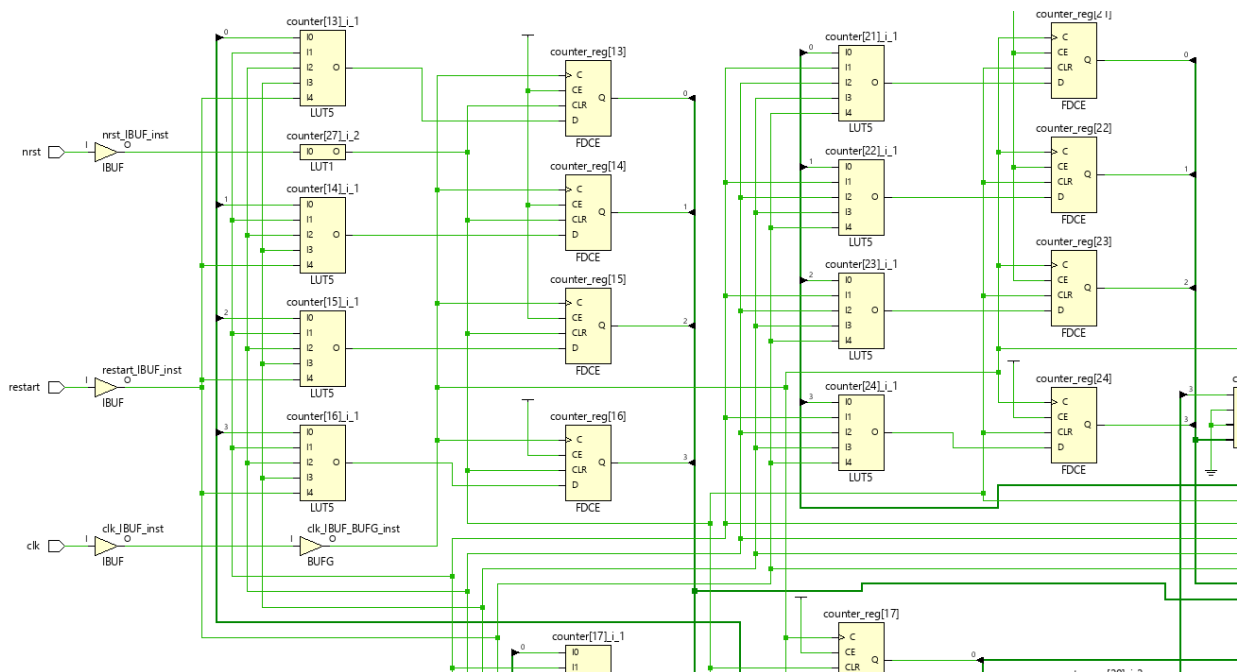
13. Exécutez la synthèse puis ouvrez la schématique. Identifiez sur la schématique les différents éléments de votre architecture RTL :

Running synth_design Cancel

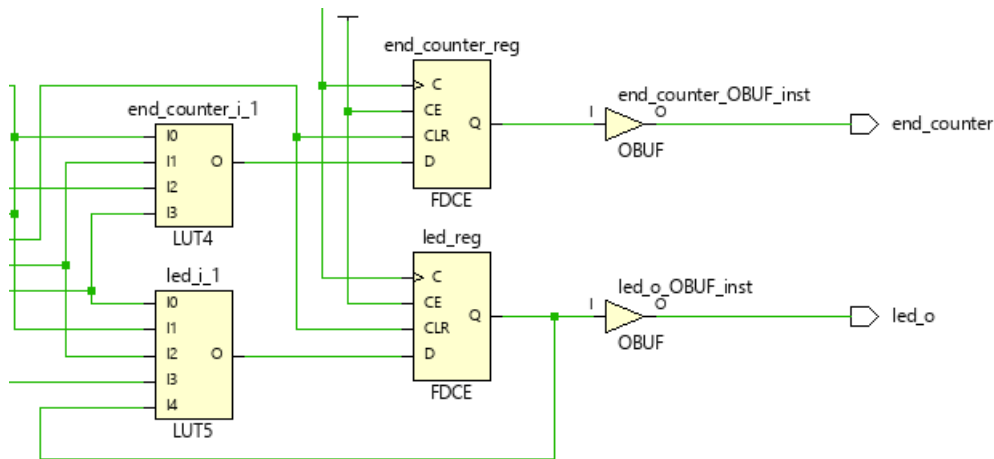
Synthesis Complete



Si je zoom sur un point en particulier :



Je vois bien tous les registres : counter_reg[0], counter_reg[1], counter_reg[2], ..., counter_reg[27] qui montre que notre compteur est constitué de 28 bascules D ce qui est primordiale pour mémoriser la valeur de notre compteur à un instant t . Chaque bit est construit selon le même principe : une LUT5 calcule la prochaine valeur du bit, puis une bascule FDCE la mémorise au front d'horloge.



De plus, j'observe bien mes 3 entrées nrst, clk et restart ainsi que les 2 sorties led_o et end_compteur :

On remarque que led_o, qui est la sortie de notre entity, prend bien sa donnée via le registre led_reg qui contient notre signal interne led que nous lui avons affecté :

```

signal counter: unsigned(27 downto 0);
signal led_s: std_logic;
signal end_counter_s: std_logic;

begin

led_o <= led_s;

```

14. Ouvrez le rapport de synthèse et relevez les ressources utilisées. Comparez vos résultats avec les résultats attendu selon votre architecture RTL.

```

-----
Start RTL Component Statistics
-----
Detailed RTL Component Info :
+---Adders :
      2 Input   28 Bit      Adders := 1
+---Registers :
      28 Bit    Registers := 1
      1 Bit     Registers := 2
+---Muxes :
      2 Input   28 Bit      Muxes := 1
      2 Input   1 Bit       Muxes := 1
-----
Finished RTL Component Statistics
-----

```

- 1 additionneur 28 bits : Permet d'additionner la valeur du compteur sur 28 bits avec +1
- 1 registre 28 bits : La valeur de compteur stocké dans un registre 28 bits.
- 2 petits registres 1 bit : probablement liés au reset ou à de la logique de contrôle interne (comparaison : if counter = MAXCOUNT).
- Multiplexeur: utilisés pour choisir entre reset (0) et incrémentation.

On retrouve bien dans notre architecture RTL le registre 28 bits, l'additionneur et le comparateur géré par les MUX.

Je m'intéresse ensuite au rapport d'analyse de puissance :

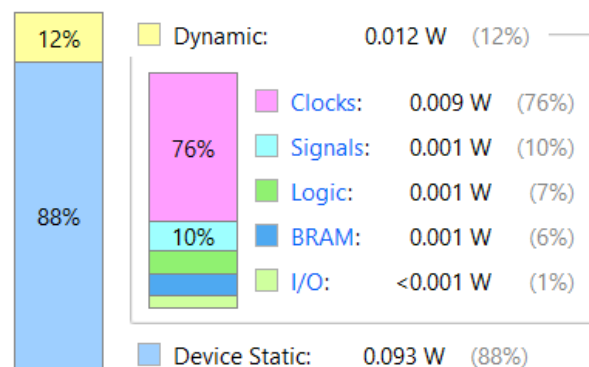
Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 0.105 W
Design Power Budget: Not Specified
Power Budget Margin: N/A
Junction Temperature: 26,2°C
Thermal Margin: 58,8°C (5,0 W)
Effective θ_{JA} : 11,5°C/W
Power supplied to off-chip devices: 0 W
Confidence level: Medium

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

On-Chip Power



-Totale on-chip Power : 0,105 W => c'est très faible, cohérent avec un simple compteur d'horloge sur un petit design comme le mien.

-Junction Temperature :26,2 °C, avec une marge thermique de 58,8 °C => aucun souci, je pense qu'on est très confortable ici.

-Détail de la puissance dynamique (0,012 W)

Horloges : 76 % — c'est le poste dominant, cela semble logique pour un compteur qui bascule à chaque cycle sur 28 bits.

Signaux : 10 %, Logique : 7 %, BRAM : 6 %, I/O : moins de 1 %.

15. Ouvrez le Set Up Debug. Placez des sondes sur les signaux à observer que vous avez défini à la question 12

clk, nrst, restart, counter, end_counter et led



Nous avons bien notre ILA dans IP sources:

```
✓ IP (1)
  > ila_0 (38)
```

Nous l'instancions ensuite dans notre description comportementale et nous l'instancions :

```
    )    ila_0a : ila_0
        PORT MAP (
            clk      => clk,
            probe0(0) => nrst,
            probe1(0) => restart,
            probe2    => std_logic_vector(counter),
            probe3(0) => end_counter_s,
            probe4(0) => led_s
        )
    );
```

16. Lancez l'implémentation puis étudiez le rapport de timing (vérifiez les violations de set up et hold et identifiez le chemin critique).

Implementation Complete 

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 4,307 ns	Worst Hold Slack (WHS): 0,029 ns	Worst Pulse Width Slack (WPWS): 3,750 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 4095	Total Number of Endpoints: 4079	Total Number of Endpoints: 2340
All user specified timing constraints are met.		

«All user specified timing constraints are met»!

On a un TNS et THS nul donc tout est bon

Le chemin critique est indiqué par la Worst Negative Slack, ça correspond à la marge temporelle la plus faible. Quand je regarde le détail, je remarque qu'il s'agit de ce chemin :

Name	Slack	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception	Clock Uncertainty
Path 21	3.798	3	17	<hidden>	<hidden>	5.261	0.915	4.346	10.0	sys_clk_pin	sys_clk_pin		0.035
Path 22	3.798	3	17	<hidden>	<hidden>	5.261	0.915	4.346	10.0	sys_clk_pin	sys_clk_pin		0.035
Path 23	3.938	3	17	<hidden>	<hidden>	5.215	0.915	4.300	10.0	sys_clk_pin	sys_clk_pin		0.035
Path 24	3.938	3	17	<hidden>	<hidden>	5.215	0.915	4.300	10.0	sys_clk_pin	sys_clk_pin		0.035
Path 25	4.095	4	17	<hidden>	<hidden>	5.729	1.445	4.284	10.0	sys_clk_pin	sys_clk_pin		0.035
Path 26	4.147	4	17	<hidden>	<hidden>	5.675	1.242	4.433	10.0	sys_clk_pin	sys_clk_pin		0.035
Path 27	4.164	4	17	<hidden>	<hidden>	5.659	1.242	4.417	10.0	sys_clk_pin	sys_clk_pin		0.035

On remarque que sur 5,261 ns de délai total, 4,346 ns (soit 83%) viennent du routage (Net Delay) contre seulement 0,915 ns de logique pure (Logic Delay). Ça signifie que le "coût" de ce chemin vient principalement de la distance physique parcourue sur le FPGA, pas de la logique.

Voici ce qu'on observe dans le détail :

Summary	
Name	Path 21
Slack	3.798ns
Source	<hidden>
Destination	<hidden>
Path Group	sys_clk
Path Type	Setup
Requirement	10.000
Data Path Delay	5.261ns (logic 0.915ns (17.391%) route 4.346ns (82.609%))
Logic Levels	3 (LUT2=1 LUT6=2)
Clock Path Skew	-0.174ns
Clock U...tainty	0.035ns

Slack Equation

Required Time - Arrival Time

Required Time 14.377ns

Arrival Time 10.579ns

II FDRE clocked by sys_clk_pin (rise@0.000ns fall@5.000ns period=10.000ns)

II FDRE clocked by sys_clk_pin (rise@0.000ns fall@5.000ns period=10.000ns)

sys_clk_pin rise@0.000ns)

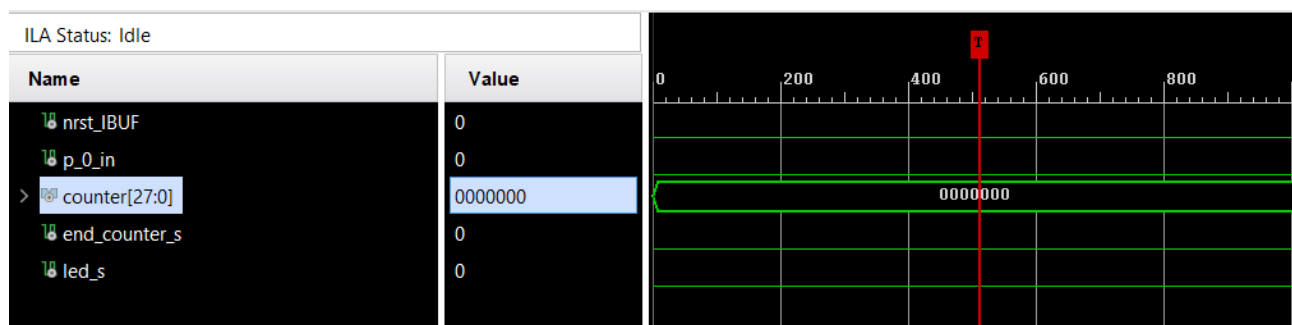
Donc notre signal arrive en 10,579ns pour un temps requis de 14,377ns ce qui lui laisse donc effectivement 3,798ns de marge. La source et la destination sont marqués en « hidden »

J'essaye de voir plus en détail le chemin complet :

Data Path				
Delay Type	Incr (ns)	Path (...)	Location	Netlist Resourc...
<hidden>	(r) 0.518	5.836	Site: SLICE_X12Y6	<hidden>
<hidden>	1.398	7.234		<hidden>
<hidden>	(r) 0.124	7.358	Site: SLICE_X18Y7	<hidden>
<hidden>	0.915	8.273		<hidden>
<hidden>	(r) 0.124	8.397	Site: SLICE_X25Y8	<hidden>
<hidden>	1.337	9.733		<hidden>
<hidden>	(r) 0.149	9.882	Site: SLICE_X26Y17	<hidden>
<hidden>	0.697	10.579		<hidden>
<hidden>			Site: SLICE_X32Y16	<hidden>

17. Générez le bitstream pour observer le système sur carte et le résultat de la ILA.

write_bitstream Complete ✓

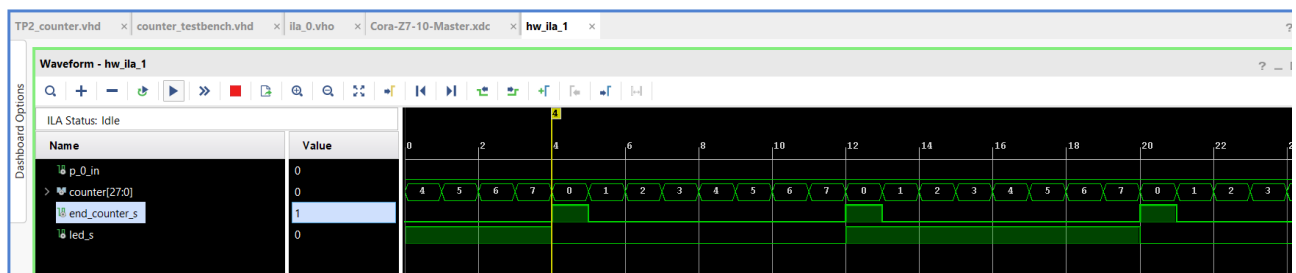


Rien ne se passe sur la carte, de plus j'observe aucun signal sur mon ILA, le compteur ne s'incrémente même pas.

Je remarque que mon nrst est à l'état bas tout le temps ce qui ne lance probablement jamais la suite du programme, je passe alors à un reset classique, je suppose alors que le bouton auquel il est associé est à l'état bas par défaut, ce qui active mon reset négatif tout le temps et donc empêche aux signaux de s'activer.

Avec le reset classique je vois les LEDs s'allumer, **MAIS en continue !**

Je regarde l'ILA :



Évidemment, je comprends que j'ai oublié le changement de la valeur maximum à laquelle le compteur doit se réinitialiser, j'ai mis 7dec (« 0111 ») comme pour le testbench.

J'en déduis donc que la LED alternait d'état tous les 7 coups d'horloge à 100Mhz ce qui paraît allumé en continu à l'oeil nu.

Je mets donc à jour cette valeur dans ma description matérielle :

```
entity counter is
  generic (
    MAX_COUNT : unsigned(27 downto 0) := "1011111010111100000111111111" --7dec
  );
```

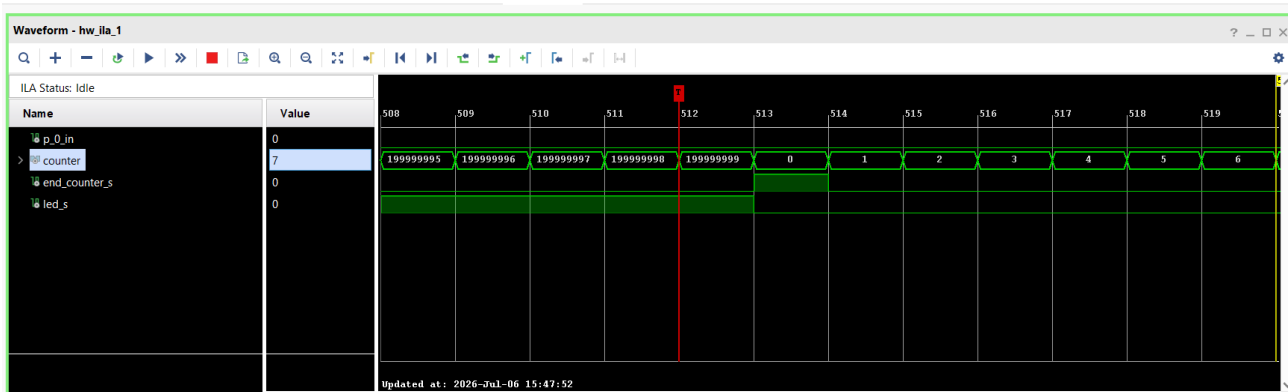
Cette fois-ci le compteur doit se réinitialiser à « 1011111010111100000111111111 » bin = 199 999 999 dec = 2 secondes (à 100Mhz).

J'observe bien que la LED s'allume et s'éteint toutes les 2 secondes

Trigger Setup - hw_ila_1					
Capture Setup - hw_ila_1					
Name	Operator	Radix	Value	Port	Comparator Usa
counter	==	[B]	1011_1110_1011_11_0	probe2[27:0]	1 of 1

Maintenant je vérifie sur l'ILA en plaçant un trigger où je veux regarder, en l'occurrence la valeur max de mon compteur en binaire («1011111010111100000111111111») pour voir ce qu'il se passe au bout de 2 secondes

Voilà ce que j'observe :



On observe bien qu'une fois que le compteur atteint la valeur maximale : $199\,999\,999 = 2$ secondes, la LED (qui était à l'état haut donc allumée) passe à l'état bas (et donc s'éteint). De plus on a bien le signal `end_counter_s` qui indique que notre compteur a atteint sa valeur maximale !

Je me suis rendu compte à quel point l'ILA a été essentielle pour le débogage et la validation de la conception directement sur la carte FPGA.

Contrairement à une simple observation des LED, qui ne fournit qu'une information limitée, cette étape de test à l'aide de l'ILA m'a permis de visualiser en temps réel les signaux internes du circuit.

Il a ainsi été possible de vérifier que le compteur s'incrémente bien, d'identifier le problème lié au reset négatif, de constater que la valeur maximale du compteur était restée configurée pour la simulation et de confirmer que le signal `end_counter_s` était bien généré lorsque le compteur atteignait sa valeur maximale.

Cette étape avec l'ILA m'a permis de comprendre qu'il constitue un outil particulièrement précieux pour analyser le comportement d'un système numérique implanté sur FPGA et faciliter le diagnostic des erreurs de conception.

Pour finir voici l'étape de démonstration :

<https://youtube.com/shorts/u9MPZ58cMDY?feature=share>